CSCE 1030: Project 1
Due: 11:59 PM on Friday, April 24, 2025

PROGRAM DESCRIPTION

In this project, students will design and implement a console-based Bank Account Simulator in C++ that models a simplified banking system. The program uses structs to organize data: an Account struct stores account details (holder name, balance, account type, PIN status, hashed PIN, and transaction history), and a Transaction struct stores individual transaction records (type, amount, and memo). Users create an account, select an account type—Checking, Savings, or Student—manage a balance, and perform PIN-protected deposits and withdrawals with robust input validation.

The three account types embody distinct, realistic banking rules: a Checking account permits withdrawals beyond the current balance but applies a fixed overdraft fee; a Savings account allows withdrawals only when sufficient funds exist, preventing overdrafts entirely; and a Student account mirrors the conservative, low-risk behavior of Savings by disallowing overdrafts, making it suitable for beginner use.

All transaction activity (deposits, withdrawals, and fees) is stored using dynamic memory (a dynamically allocated array of Transaction structs) and can grow as new transactions are added. Data is persisted using file I/O in a comma-separated values (CSV) format, with one file per user (<holder>.txt), allowing account information and transaction history to be saved and loaded across program runs.

PROGRAM REQUIREMENTS

Banner

As with all projects in this course, your program's output will display your name, your EUID, your e-mail address, the department name, and course number. This means that your program will print this information to the terminal (see the sample output).

Build a console-based Bank Account Simulator that manages:

- **Account holder name** — letters, digits, and spaces only; validated using isalpha, isdigit, or isspace.
- **Account type** — CHECKING, SAVINGS, or STUDENT (selected by entering 1–3).
- **Balance** — a double value representing account funds.
- **PIN** — 4–6 digits; stored as a **hashed value**; required for Deposit/Withdraw.
- **Transaction history** — stored as a **dynamically allocated array of Transaction structs**, each with:
  - Type: 'D' (Deposit), 'W' (Withdraw), 'F' (Fee)
  - Amount: double
  - Memo: string
- **File storage** — Each user has a **separate CSV file** <holder>.txt. Account info (holder, balance, type, PIN hash, PIN set) and all transactions are saved and loaded on program start.

Transaction Storage and Consistency Rules

- Transaction history uses a dynamically allocated array of Transaction structs.
- When adding a transaction, the array is reallocated to include the new entry.

- **Consistency Rule:** the type, amount, and memo in each Transaction struct always belong together.
- All changes (deposit, withdrawal, fees) are **saved to the user's file immediately**.

Policy differences by account type

- **Checking** — allows overdrafts. If withdrawal would cause negative balance, a $35 fee is applied and recorded as a 'F' transaction.
- **Savings / Student** — withdrawals that would go negative are denied; no changes recorded.

Example of stored data

For a user "Alice Johnson":

- **Struct content:**
  holder = "Alice Johnson"
  balance = 100.0
  type = CHECKING
  pinHash = 123456789 (hashed PIN)
  transactions[0] = { 'D', 50.0, "Paycheck" }

- **CSV file content (Alice Johnson.txt):**
  Alice Johnson,100.0,1,1,123456789
  1
  D,50.0,Paycheck

Required Features & Flow

1. **Header** — Print a banner showing program name and basic info (name, EUID, email, department, course number).
2. **Name input** — Read with getline; re-prompt until only letters, digits, or spaces (checked with isalpha/isdigit/isspace).
3. **Initial balance** — Read a double from the user.
4. **Account type selection** — Prompt for 1..3; validate with do/while; map to AccountType enum (CHECKING, SAVINGS, STUDENT).
5. **Main loop (inline menu + switch)** — Display 6 options and handle the choice via switch:
   1. Deposit (PIN required)
   2. Withdraw (PIN required)
   3. Show Account Details
   4. Set/Change PIN (4–6 digits; hashed for security)
   5. View Transactions by Type (D/W/F)
   6. Exit
6. **PIN enforcement** — For Deposit/Withdraw:
   - If no PIN exists, force creation using setOrChangePin() (validate length & digits; confirm).
   - Store PIN as a hashed value in the account struct.
   - When checking PIN, compare the hash; if incorrect, cancel the operation.

7. **Deposit** — Read amount (double) -> cin.get() -> memo via getline; increase balance; record a 'D' transaction. Transactions are stored in a **dynamically allocated array of Transaction structs**, and the account is **saved to a CSV file**.
    - Example: deposit $50 with memo "Paycheck".
8. **Withdraw** — Read amount (double) -> cin.get() -> memo via getline; apply account-type rules:
    - If sufficient funds: subtract amount, record 'W' transaction.
    - If CHECKING and insufficient: allow overdraft, apply $35 fee, record 'W' and 'F' transactions (memo "Overdraft Fee").
    - If SAVINGS/STUDENT and insufficient: deny withdrawal, no changes recorded.
    - All updates are **saved to the user's CSV file**.
9. **Show Details** — Print holder name, account type, balance, and PIN status (set/not set). Show up to 10 most recent transactions from newest to oldest.
    - Example: [Deposit] $50 | Paycheck.
10. **View by Type** — Ask for D/W/F (case-insensitive); print matching transactions from newest to oldest; if none, print (No transactions of that type).
11. **File persistence** — Each user has a **separate CSV file** named <holder>.txt. Account info (holder, balance, type, PIN hash, PIN set) and all transactions are saved. On startup, if a file exists for the holder, load account info and transaction history.
12. **Dynamic memory management** — Transaction history is stored as a dynamically allocated array of Transaction structs. Memory is **reallocated each time a new transaction is added** to ensure all transactions are stored in order.
13. **Example of stored data** — For an account "Alice Johnson" with balance 100.0 and a deposit:

- Struct content:
    - holder = "Alice Johnson"
    - balance = 100.0
    - type = CHECKING
    - pinHash = 123456789 (hashed PIN)
    - transactions[0] = { 'D', 50.0, "Paycheck" }
- CSV file content (Alice Johnson.txt):

```
Alice Johnson,100.0,1,1,123456789
1
D,50.0,Paycheck
```

Function Specifications

1) void printHeader()

- Prints program banner including student name, EUID, email, department, and course number.
- Called once at program start.

---

2) bool isValidName(const string& name)

- Iterates through each character in the string.
- Returns true only if every character is a letter, digit, or space using isalpha, isdigit, isspace.
- Stops early if an invalid character is found.
- Example: "John Doe" → true, "John_Doe!" → false

---

3) string readValidName()

- Prompts user for name using getline.
- Calls isValidName to validate input.
- Re-prompts until valid input is entered.
- Returns the validated name string.
- Example: User enters "John!" → rejected → enters "John Doe" → accepted

---

4) bool isValidPin(const string& p)

- Checks that length is between 4 and 6 characters.
- Ensures all characters are digits using isdigit.
- Returns true only if both conditions are satisfied.
- Example: "1234" → true, "12a4" → false

---

5) bool setOrChangePin(Account& acc)

- Prompts user to enter a PIN and confirm it.
- Validates PIN using isValidPin.
- If valid and both entries match:
    - Hashes the PIN using hashPin()
    - Stores it in acc.pinHash
    - Sets acc.pinSet = true
- If mismatch, prints error and returns false.
- Example: "1234" + "1234" → PIN stored as hash

---

6) bool requirePin(Account& acc)

- Ensures a valid PIN is entered before sensitive actions.
- If no PIN is set:
    - Prompts user to create one via setOrChangePin.
- If PIN exists:
    - Prompts user for PIN input
    - Hashes input and compares with acc.pinHash
- Returns true if correct, false otherwise.

---

7) AccountType chooseAccountType()

- Prompts user to enter 1–3.
- Uses loop to repeat until valid input is given.
- Maps input to enum values:
    - 1 → CHECKING
    - 2 → SAVINGS
    - 3 → STUDENT

- Example: Input 2 → returns SAVINGS

---

8) string accountTypeToString(AccountType t)

- Converts enum value into readable string.
- Used for display purposes in output.
- Example: CHECKING → "Checking"

---

9) void recordTransaction(Account& acc, char type, double amount, const string& memo)

- Dynamically resizes the transaction array:
    1. Allocates a new array of size txnCount + 1
    2. Copies existing transactions into new array
    3. Deletes old array using delete[]
    4. Adds new transaction at last index
- Updates txnCount.
- Calls saveToFile() to persist changes.
- Example: Adds {type='D', amount=100, memo="Gift"}

---

10) void printRecentTransactions(const Account& acc, int maxToShow=10)

- Displays transactions from newest to oldest.
- Determines how many to show (min(txnCount, maxToShow)).
- Formats output with labels [Deposit], [Withdraw], [Fee].
- If no transactions, prints (No transactions yet).
- Example Output:
  [Deposit] $100 | Gift

---

11) void viewTransactionsByType(const Account& acc)

- Prompts user to enter D, W, or F.
- Converts lowercase input to uppercase manually.
- Iterates through all transactions and prints only matches.
- Tracks if any were printed; if none, prints message.
- Example: Input 'd' → prints all deposits

---

12) void deposit(Account& acc)

- Calls requirePin to verify user.
- Prompts for deposit amount.
- Uses cin.ignore() + getline() to read multi-word memo.
- Adds amount to balance.
- Calls recordTransaction with type 'D'.
- Prints updated balance.
- Example: Deposit $200 → balance increases and transaction stored

---

13) void withdraw(Account& acc)

- Calls requirePin to verify user.
- Reads withdrawal amount and memo.
- Computes newBal = acc.balance - amount.

Logic:

- If newBal >= 0:
    - Deduct amount
    - Record 'W' transaction
- If newBal < 0 and CHECKING:
    - Allow overdraft

    - Apply $35 fee
    - Record 'W' and 'F' transactions
- If newBal < 0 and SAVINGS/STUDENT:
    - Deny withdrawal
- Example: Balance $50, withdraw $70 (CHECKING) → new balance = 50 - 70 - 35 = -55

---

14) void showDetails(const Account& acc)

- Displays full account summary:
    - Holder name
    - Account type
    - Balance
    - PIN status (set)/(not set)
- Calls printRecentTransactions to show history.

---

15) void loadFromFile(Account& acc)

- Opens file <holder>.txt using ifstream.
- Reads first line (account info) using getline with delimiter ','.
- Converts strings to appropriate types (stod, stoi, stoul).
- Reads transaction count.
- Allocates dynamic array: new Transaction[txnCount].
- Reads each transaction line and stores into struct array.
- Example:
  File:

```
John Doe,1500.0,1,1,987654
2
D,500,Paycheck
W,50,Groceries
```

→ Creates acc.transactions[0] and [1] with corresponding values.

---

16) void saveToFile(const Account& acc)

- Opens <holder>.txt using ofstream.
- Writes:
    1. Account info (comma-separated)
    2. Transaction count
    3. Each transaction line (type,amount,memo)
- Overwrites file each time to keep data consistent.
- Example Output:

```
John Doe,1500.0,1,1,987654
2
D,500,Paycheck
W,50,Groceries
```

---

17) unsigned long hashPin(const string& pin) **(This function code is given to you)**

- Computes hash using formula:
  h = h * 31 + c for each character
- Produces numeric representation of PIN.
- Used for secure comparison instead of storing raw PIN.
- Example: "1234" → some numeric hash value

---

18) string makeFileName(const string& holder)

- Returns filename by appending .txt to account holder name.
- Used by file I/O functions.
- Example: "John Doe" → "John Doe.txt"

INLINE MENU AND VALIDATION (IN MAIN)

- Display options 1–6 inline; read choice 1–6 with validation; dispatch with switch.

DESIGN (ALGORITHM):

On a piece of paper (or word processor), write down the algorithm, or sequence of steps, that you will use to solve the problem. You may think of this as a "recipe" for someone else to follow. Continue to refine your "recipe" until it is clear and deterministically solves the problem. Be sure to include the steps for prompting for input, performing calculations, and displaying output.
You should attempt to solve the problem by hand first (using a calculator as needed) to work out what the answer should be for a few sets of inputs.
Type these steps and calculations into a document (i.e., Word, text, or PDF) that will be submitted along with your source code. Note that if you do any work by hand, images (such as pictures) may be used, but they must be clear and easily readable. This document shall contain both the algorithm and any supporting hand-calculations you used in verifying your results.

TESTING:

Test your program to check that it operates as desired with a variety of inputs. Then, compare the answers your code gives with the ones you get from hand calculations.

SUBMISSION:

- Your program will be graded based largely upon whether it works correctly on the CSE machines, so you should make sure your program compiles and SAMPLE OUTPUTs on the CSE machines.

- Your program will also be graded based upon your program style. This means that you should use comments (as directed), meaningful variable names, and a consistent indentation style as recommended in the textbook and in class.

- We will be using an electronic homework submission on Canvas to make sure that all students hand in their programming projects on time. You will submit both (1) the program source code file and (2) the algorithm design document to the **Project 3** dropbox on Canvas by the due date and time.

- Projects are meant to be problem-solving exercises and are designed to help you practice your coding on larger projects with various pieces of functionality.

- If you choose to work alone, the coding should be primarily your sole work.

- If you choose to work in a group, you are allowed to get assistance from group members when working on these assignments. However, each student is required to report the name(s) of the students they worked with on the assignment. The maximum size of the group is 4 students, you can of course form a smaller group.

- Cheating for these assignments is defined as copying from a fellow student outside your working group, or copying from the web, or any other sources outside the class. You should not copy

someone else's code or let a classmate examine your code if you have not identified as working in a group for your homework.

- As a safety precaution, do not edit your program (using vim or nano) after you have submitted your program where you might accidentally re-save the program, causing the timestamp on your file to be later than the due date. If you want to look (or work on it) after submitting, make a copy of your submission and work on that copy. Should there be any issues with your submission, this timestamp on your code on the CSE machines will be used to validate when the program was completed.

Sample Output 1 — New Account Creation and Deposit

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
================ Bank Account Simulator ================

Enter account holder name: John Doe
Enter initial balance: $200.00
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 1

-------------------------------------------------------------
Account Holder: John Doe
Account Type:   Checking
Balance:        $200.00
-------------------------------------------------------------
1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN
5) View Transactions by Type
6) Exit
Select an option (1-6): 1

No PIN set. Create one to continue.
Set PIN (4-6 digits): 1234
Confirm PIN: 1234
PIN set.
Enter deposit amount: $150
Enter memo: Paycheck
Deposited $150.00. New balance: $350.00
```

Sample Output 2 — Withdraw With Insufficient Funds for Savings

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+

================ Bank Account Simulator ================

Enter account holder name: Jane Smith
Enter initial balance: $100.00
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 2

-------------------------------------------------------------
Account Holder: Jane Smith
Account Type:   Savings
Balance:        $100.00
-------------------------------------------------------------
1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN
5) View Transactions by Type
6) Exit
Select an option (1-6): 2

No PIN set. Create one to continue.
Set PIN (4-6 digits): 5678
Confirm PIN: 5678
PIN set.
Enter withdrawal amount: $150
Enter memo: Emergency
Withdrawal denied. Insufficient funds.
```

SAMPLE OUTPUT 3 — Checking Overdraft Fee Applied

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
```

```
================ Bank Account Simulator ================

Enter account holder name: Bob Lee
Enter initial balance: $50.00
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 1

------------------------------------------------------------
Account Holder: Bob Lee
Account Type:   Checking
Balance:        $50.00
------------------------------------------------------------
1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN
5) View Transactions by Type
6) Exit
Select an option (1-6): 2

No PIN set. Create one to continue.
Set PIN (4-6 digits): 1111
Confirm PIN: 1111
PIN set.
Enter withdrawal amount: $70
Enter memo: Rent
Overdraft! Fee $35.00 applied. New balance: $-55.00
```

SAMPLE OUTPUT 4 — Show Account Details and Recent Transactions

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
================ Bank Account Simulator ================

Enter account holder name: Alice Green
Enter initial balance: $500.00
Choose account type:
1) Checking
2) Savings
3) Student
```

```
Enter (1-3): 1

------------------------------------------------------------
Account Holder: Alice Green
Account Type:   Checking
Balance:        $500.00
------------------------------------------------------------
1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN
5) View Transactions by Type
6) Exit
Select an option (1-6): 1

No PIN set. Create one to continue.
Set PIN (4-6 digits): 2222
Confirm PIN: 2222
PIN set.
Enter deposit amount: $200
Enter memo: Salary
Deposited $200.00. New balance: $700.00

Select an option (1-6): 2
Enter PIN: 2222
Enter withdrawal amount: $100
Enter memo: Groceries
Withdrew $100.00. New balance: $600.00

Select an option (1-6): 3
================ Account Details ================
Holder:  Alice Green
Type:    Checking
Balance: $600.00
PIN:     (set)
---------------- Recent Transactions ----------------
[Withdraw] $100.00 | Groceries
[Deposit ] $200.00 | Salary
--------------------------------------------------
==================================================
```

SAMPLE OUTPUT 5 — View Transactions by Type (Error & Success)

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
```

```
|         Student Name EUID euid@my.unt.edu       |
+-----------------------------------------------+
================ Bank Account Simulator ================

Enter account holder name: Carlos Vega
Enter initial balance: $300.00
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 1

------------------------------------------------------------
Account Holder: Carlos Vega
Account Type:   Checking
Balance:        $300.00
------------------------------------------------------------
1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN
5) View Transactions by Type
6) Exit
Select an option (1-6): 1

No PIN set. Create one to continue.
Set PIN (4-6 digits): 3333
Confirm PIN: 3333
PIN set.
Enter deposit amount: $100
Enter memo: Bonus
Deposited $100.00. New balance: $400.00

Select an option (1-6): 5
Show which transactions? (D/W/F): W
(No transactions of that type)

Select an option (1-6): 2
Enter PIN: 3333
Enter withdrawal amount: $50
Enter memo: Snacks
Withdrew $50.00. New balance: $350.00

Select an option (1-6): 5
Show which transactions? (D/W/F): w
[Withdraw] $50.00 | Snacks
```

SAMPLE OUTPUT 6 — Account Reload From File, Hashed PIN, Dynamic Transactions

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
================ Bank Account Simulator ================

Enter account holder name: Sarah Connor
Enter initial balance: $0
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 2

No saved data for Sarah Connor.

-------------------------------------------------------------
Account Holder: Sarah Connor
Account Type:   Savings
Balance:        $0.00
-------------------------------------------------------------
1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN
5) View Transactions by Type
6) Exit
Select an option (1-6): 1

No PIN set. Create one to continue.
Set PIN (4-6 digits): 4321
Confirm PIN: 4321
PIN set.
Enter deposit amount: $500
Enter memo: Initial Deposit
Deposited $500.00. New balance: $500.00

Select an option (1-6): 2
Enter PIN: 4321
Enter withdrawal amount: $200
Enter memo: Groceries
Withdrew $200.00. New balance: $300.00
```

```
Select an option (1-6): 3
================ Account Details ================
Holder:  Sarah Connor
Type:    Savings
Balance: $300.00
PIN:     (set)
---------------- Recent Transactions ----------------
[Withdraw] $200.00 | Groceries
[Deposit ] $500.00 | Initial Deposit
--------------------------------------------------
==================================================

Select an option (1-6): 6
Saving account to Sarah Connor.txt ...
Goodbye!
```

SAMPLE OUTPUT 7 — Normal Savings Account Session

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
================ Bank Account Simulator ================

Enter account holder name: John Doe
Enter initial balance: $500
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 2

--------------------------------------------------------------
Account Holder: John Doe
Account Type:   Savings
Balance:        $500.00
--------------------------------------------------------------
1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN
5) View Transactions by Type
6) Exit
Select an option (1-6): 1
```

```
No PIN set. Create one to continue.
Set PIN (4-6 digits): 1234
Confirm PIN: 1234
PIN set.

Enter deposit amount: $200
Enter memo: Paycheck
Deposited $200.00. New balance: $700.00

------------------------------------------------------------
Select an option (1-6): 2

Enter PIN: 1234
Enter withdrawal amount: $100
Enter memo: Grocery
Withdrew $100.00. New balance: $600.00

------------------------------------------------------------
Select an option (1-6): 3
================ Account Details ================
Holder:  John Doe
Type:    Savings
Balance: $600.00
PIN:     (set)
---------------- Recent Transactions ----------------
[Withdraw] $100.00 | Grocery
[Deposit ] $200.00 | Paycheck
-----------------------------------------------------

------------------------------------------------------------
Select an option (1-6): 6

Thank you for using the Bank Account Simulator. Goodbye!
```

SAMPLE OUTPUT 8 — Checking Account with Overdraft

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
Enter account holder name: Alice Smith
Enter initial balance: $50
Choose account type:
1) Checking
2) Savings
```

```
3) Student
Enter (1-3): 1

------------------------------------------------------------
Account Holder: Alice Smith
Account Type:   Checking
Balance:        $50.00
------------------------------------------------------------
1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN
5) View Transactions by Type
6) Exit
Select an option (1-6): 2

No PIN set. Create one to continue.
Set PIN (4-6 digits): 5678
Confirm PIN: 5678
PIN set.

Enter PIN: 5678
Enter withdrawal amount: $100
Enter memo: Emergency Cash
Overdraft! Fee $35.00 applied. New balance: $-85.00

------------------------------------------------------------
Select an option (1-6): 3
================ Account Details ================
Holder:  Alice Smith
Type:    Checking
Balance: $-85.00
PIN:     (set)
---------------- Recent Transactions ----------------
[  Fee  ] $35.00 | Overdraft Fee
[Withdraw] $100.00 | Emergency Cash
-----------------------------------------------------
```

SAMPLE OUTPUT 9 — Incorrect PIN Attempt

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
Enter account holder name: Bob Lee
```

```
Enter initial balance: $300
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 3

------------------------------------------------------------
Select an option (1-6): 2

No PIN set. Create one to continue.
Set PIN (4-6 digits): 4321
Confirm PIN: 4321
PIN set.

Enter PIN: 1234
Incorrect PIN.
Deposit cancelled.

Enter PIN: 4321
Enter deposit amount: $50
Enter memo: Gift
Deposited $50.00. New balance: $350.00
```

SAMPLE OUTPUT 10 — Transaction View by Type

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
------------------------------------------------------------
Select an option (1-6): 5
Show which transactions? (D/W/F): d
[Deposit ] $50.00 | Gift
[Deposit ] $300.00 | Initial Balance
------------------------------------------------------------
Select an option (1-6): 5
Show which transactions? (D/W/F): w
(No transactions of that type)
```

SAMPLE OUTPUT 11 — Invalid Name & File Persistence

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
```

```
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
Enter account holder name: @Charlie!
Name can only contain letters, digits, and spaces. Try again: Charlie
Brown
Enter initial balance: $1000
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 1

------------------------------------------------------------
Select an option (1-6): 6
Thank you for using the Bank Account Simulator. Goodbye!
```